

CSCE 614 COMPUTER ARCHITECTURE · MODULE 13 · LECTURE NOTES

Energy, Scaling, and Specialization

Why the free lunch ended, and what replaced it.

MODULE OUTCOMES

Explain Dennard scaling and why its end changed everything.

Explain dark silicon and the shift to multicore.

Explain why specialised hardware is more efficient.

Describe the current landscape of accelerators.

Build a working cost model of a modern machine.

1 The free lunch, and why it ended

From roughly 1975 to 2005, programs became faster each year without being modified. Clock frequencies rose, and single-thread performance rose with them. Two separate trends made this possible, and they are routinely conflated.

	Moore's law	Dennard scaling
Statement	Transistor density doubles roughly every two years.	As transistors shrink, voltage and current scale down proportionally, so power density stays constant.
Gives you	More transistors.	The ability to run them faster at the same power.
Status	Slowed, but continued well past 2005.	Ended around 2005.

$$P = C \cdot V^2 \cdot f$$

Dynamic power depends on capacitance, the *square* of voltage, and frequency. Dennard's insight was that shrinking a transistor reduces C and permits a lower V, so frequency could rise with no net power increase. Smaller, faster, same power, for thirty years.

WHY VOLTAGE STOPPED SCALING

Lowering supply voltage requires lowering the transistor's threshold voltage. Below a certain point, transistors no longer switch off cleanly, and **leakage current** flows continuously even when nothing is switching.

Leakage rises exponentially as threshold voltage falls, so beyond about 2005 further voltage reduction cost more in static power than it saved in dynamic power.

With V fixed and C still falling, power *density* rises as chips shrink. Frequency became limited by heat removal rather than by transistor switching speed. Clocks flattened at 3–4 GHz and have stayed there for two decades.

Dark silicon is the consequence: a chip can contain more transistors than it can power simultaneously, so some fraction must always be idle or run slowly.

2 Three responses

Response	What it does	What it costs
Multicore	Use the transistor budget for several simpler cores instead of one larger one. Total throughput rises even though per-core speed does not.	The burden moves to software. Only parallel programs benefit, and Amdahl's law (Module 01) caps the gain.
Heterogeneity	Combine a few large high-performance cores with many small efficient ones, scheduling work according to need.	Scheduling complexity and variable, less predictable performance. Benchmarking becomes harder.
Specialisation	Dedicate silicon to specific tasks: video codecs, matrix units, cryptography, ray traversal.	Only helps workloads that match. Inflexible, and obsolete if the algorithm changes.

WHAT THIS MEANT FOR PROGRAMMERS

Before 2005, performance was the hardware's problem. A program written in 1995 ran several times faster in 2000 with no changes.

After 2005, additional performance came almost entirely from parallelism, and parallelism must be programmed. Single-thread performance has improved perhaps two to three times in twenty years, all of it from microarchitecture — wider issue, better prediction, larger caches — rather than frequency.

This is why parallel programming moved from a specialist concern to a general one, and why Amdahl's law is now a practical constraint on ordinary software rather than a theoretical observation about supercomputers. CSCE 735 exists because of this transition.

3 Why specialisation wins

A fixed-function unit can be hundreds of times more energy-efficient than a general-purpose core performing the same computation. The reason is where the energy actually goes.

Operation	Approximate energy
32-bit integer add	< 1 pJ
32-bit floating-point multiply-add	a few pJ
Reading a 32-bit value from a register file	a few pJ
Fetching, decoding, renaming, scheduling, and retiring one instruction	tens to hundreds of pJ
Reading 32 bits from DRAM	hundreds to thousands of pJ

THE ARITHMETIC IS ALMOST FREE; EVERYTHING AROUND IT IS NOT

In a general-purpose out-of-order core, the actual computation is a small fraction of the energy consumed. The majority goes to instruction supply, dependence tracking, scheduling, and data movement.

Fixed-function hardware eliminates all of it. There is no instruction to fetch or decode, no renaming, no scheduling; data flows through dedicated logic laid out for the specific computation, with operands travelling minimal distances.

That is the source of the 10–1000× efficiency advantage, and it explains why every phone contains a dozen accelerators: under a fixed power budget, specialisation is the only remaining way to increase useful work.

Accelerator	Workload	Mechanism
GPU	Data-parallel throughput work	Die area spent on arithmetic rather than control; latency hidden by thread switching (Module 10).
Video encode/decode block	H.264, H.265, AV1	Entirely fixed function. Roughly 100× the efficiency of software decode, which is why phones play 4K video without heating up.
Tensor / matrix units	Dense matrix multiplication	Systolic arrays: operands flow between adjacent processing elements, minimising data movement — the dominant energy cost.
Ray tracing units	BVH traversal and triangle intersection	Fixed-function traversal and intersection test. This is what made real-time ray tracing viable, and it is directly relevant to CSCE 647.
NPU / DSP	Inference, signal processing	Specialised datapaths with reduced precision (int8, bf16), trading numerical range for efficiency.

THE BET SPECIALISATION MAKES

Fixed-function silicon does exactly one thing, takes years to design and fabricate, and is wasted if the algorithm changes. A decoder built for H.265 cannot decode AV1.

So specialisation is a wager that the workload will remain stable long enough to repay the investment. It has paid handsomely for video compression, rasterised graphics, and dense linear algebra, all stable for a decade or more.

It is a poor wager for anything evolving quickly — which is exactly why programmable shaders displaced fixed-function graphics pipelines in the first place, and a useful corrective to the assumption that specialisation is always the future.

4 The cost model this course was for

The point of thirteen modules was not the mechanisms individually. It was to build a model good enough to predict and diagnose. When code is slow, work down this list:

Question	What to measure	Module
Is the core computing or waiting?	IPC	01, 08
Is it missing in cache, and at which level?	L1/L2/LLC miss rates	05, 06
Is it missing in the TLB?	dTLB-load-misses	07
Is it mispredicting branches?	branch-misses rate	04
Did the hot loop vectorise?	Compiler reports; generated assembly	09
Are cores contending or falsely sharing?	perf c2c; scaling curve	11
Is the synchronisation correct and cheap?	ThreadSanitizer; contention counters	12
Would this workload suit a GPU?	Arithmetic intensity; transfer volume	10

4.1 What you should now believe

- **Memory dominates.** A 2% main-memory miss rate contributes ten times more to CPI than a 5% branch misprediction rate. Check memory first, always.
- **Latency is hidden, not eliminated.** Caches, prefetchers, out-of-order execution, hyperthreading, and GPU thread switching are all the same idea applied at different scales: find something else to do while waiting.
- **Parallelism exists at every level** — vector lanes, pipeline stages, cores, sockets, devices — and each level has its own cost model and its own failure modes.
- **Data layout decides performance** more often than algorithm choice does, once the algorithm is asymptotically reasonable. This is the single most actionable conclusion of the course.

- **Everything here is measurable.** Every claim in these thirteen modules can be checked on your own machine in an afternoon, and you should distrust any you have not checked.

WHERE THIS LEAVES YOU

You can look at a loop and predict, approximately, what will make it slow — before running it. You can take a slow program and diagnose it from hardware counters rather than by guessing. And you know precisely why CSCE 629's RAM model is wrong, in which direction, and by how much.

CSCE 641 is where this becomes frame time: a rendering loop is memory-bound far more often than it is arithmetic-bound, and the layout of vertex and texture data decides it. **CSCE 735** takes the parallelism hierarchy seriously and makes the scaling arguments quantitative. **CSCE 611** explains the operating system layered on top of all this hardware.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Why Computer Architecture? and the closing lectures	The scaling story and the case for specialisation, from someone who works on it.
Hennessy & Patterson — 'A New Golden Age for Computer Architecture' (Turing lecture, free)	The authors of the standard textbook on where architecture goes after Dennard scaling. Short, and the best single summary of this module's argument.
Herb Sutter — The Free Lunch Is Over (2005)	The essay that told the software industry the transition had happened. Worth reading as a historical document.
Mark Horowitz — Computing's Energy Problem (ISSCC keynote, free slides)	Where the energy actually goes in a modern core. The source of the numbers in §3.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure your processor's clock frequency under sustained load versus idle. Relate the difference to the power equation and to the thermal throttling discussion of Module 01.
- 2 Compare the energy consumed decoding a video in software versus using the hardware decoder, using battery drain or a power meter. Report the ratio.
- 3 Measure parallel scaling of a workload from 1 to N threads. Fit Amdahl's law to the curve and estimate the serial fraction.

- 4 Take a kernel and run it on CPU with SIMD, on CPU multithreaded, and on GPU. Report time and, if you can measure it, energy for each.
- 5 Write the Module 4 diagnostic checklist as a script that runs perf stat with the right counters and prints an assessment. You will use it afterwards.
- 6 **Project 2 is now due.** Complete the optimisation project described in the syllabus: a 5× speedup with predictions recorded before each measurement.
- 7 Write a one-page summary of your own machine: cache sizes, TLB reach, misprediction penalty, SIMD width, core count, memory bandwidth. Every number measured by you.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish Moore's law from Dennard scaling. Which one ended around 2005, and why?
- 2 Why did voltage stop scaling, and what is dark silicon?
- 3 What did the shift to multicore transfer to software, and which earlier result became practically important as a result?
- 4 Why is fixed-function hardware 10–1000× more energy efficient? Where does the energy go in a general-purpose core?
- 5 What bet does specialisation make, and give one workload where it paid and one where programmability won.
- 6 List the diagnostic questions you would ask, in order, about a slow program.
- 7 Which costs more CPI: a 2% memory miss rate or a 5% branch misprediction rate? By roughly how much?